



demo7-rest-swagger – Swagger Example

A)Develop the REST API

- 1) Copy demo1-rest-boot as demo7-rest-swagger
- 2) Delete the following
 - a. Hello.java
 - b. HelloController.java
- 3) Update pom.xml
- 4) Update application.properties

5) Write Account.java

```
public class Account {  
  
    private int accno;  
    private String atype;  
    private String bcode;  
    private double balance;  
  
    //Constrcutors  
    //Setters and Getters  
    //toString()  
}
```

6) Write AccountController.java

```
package com.jlcindia.springboot;  
  
import java.util.*;  
import org.springframework.web.bind.annotation.*;  
  
@RestController  
public class AccountController {  
  
    @GetMapping("/mybal/{accno}")  
    public double getBalance(@PathVariable("accno") String accno) {  
        System.out.println("AC-getBalance()" + accno);  
        return 25000;  
    }  
}
```



```
@GetMapping("/myaccount/{accno}")
public Account getAccountByAccno(@PathVariable("accno") int accno) {
    System.out.println("AC-getAccountByAccno() + accno);

    Account acc = new Account(accno, "SA", "B99", 29000.0);
    return acc;

}

@GetMapping("/myaccounts")
public List<Account> getAllCounts() {
    System.out.println("AC-getAllCounts()");

    List<Account> accList = new ArrayList<>();
    accList.add(new Account(101, "SA", "B99", 19000.0));
    accList.add(new Account(102, "SA", "B99", 29000.0));
    accList.add(new Account(103, "SA", "B99", 29000.0));
    return accList;
}

@PostMapping(value = "/addAccount")
public Account addAccount(@RequestBody Account acc) {
    System.out.println("AC-addAccount()");
    acc.setAccno(123);
    return acc;
}

@PutMapping(value = "/updateAccount")
public String updateAccount(@RequestBody Account acc) {
    System.out.println("AC-updateAccount()");
    String msg = "Account Updated Successfully";
    return msg;
}

@DeleteMapping(value = "/deleteAccount/{myaccno}")
public String deleteAccount(@PathVariable("myaccno") String accno) {
    System.out.println("AC-deleteAccount()");
    String msg = "Account Deleted Successfully";
    return msg;
}
}
```



7) Write Customer.java

```
package com.jlcindia.springboot;

public class Customer {
    private int cid;
    private String cname;
    private String email;
    private long phone;
    private String city;

    //Constrcutors
    //Setters and Getters
    //toString()

}
```

8) Write CustomerController.java

```
package com.jlcindia.springboot;

import java.util.*;
import org.springframework.web.bind.annotation.*;

@RestController
public class CustomerController {

    @GetMapping(value = "/mycustomer/custId/{custId}", produces =
"application/xml")
    public Customer getCustomerByCid(@PathVariable("custId") int custId) {
        System.out.println("CC-getCustomerByCid() - " + custId);
        Customer cust = new Customer(custId, "Srinivas", "sri@jlc.com", 12345, "Blore");
        return cust;
    }

    @GetMapping(value = "/mycustomer/phone/{phone}", produces =
"application/xml")
    public Customer getCustomerByPhone(@PathVariable("phone") long phone) {
        System.out.println("CC-getCustomerByPhone() - " + phone);

        Customer cust = new Customer(109, "Dande", "dande@jlc.com", phone, "Blore");
        return cust;
    }
}
```



```
@GetMapping(value = "/mycustomer/city/{city}", produces = "application/json")
public List<Customer> getCustomersByCity(@PathVariable("city") String city) {
    System.out.println("CC-getCustomersByCity() - " + city);
    List<Customer> custList = new ArrayList<>();
    custList.add(new Customer(101, "Sri", "sri@jlc.com", 111, city));
    custList.add(new Customer(102, "Vas", "vas@jlc.com", 222, city));
    custList.add(new Customer(103, "SD", "sd@jlc.com", 333, city));
    return custList;
}

}
```

B)Add Swagger to the Project

9) Add the following dependencies in pom.xml

```
<dependency>
<groupId>io.springfox</groupId>
<artifactId>springfox-swagger2</artifactId>
<version>2.4.0</version>
</dependency>

<dependency>
<groupId>io.springfox</groupId>
<artifactId>springfox-swagger-ui</artifactId>
<version>2.4.0</version>
</dependency>
```

10) Mark the Boot main class with @EnableSwagger2 annotation

```
@SpringBootApplication
@EnableSwagger2
public class MyBootApplication{
    public static void main(String args[]) {
        SpringApplication.run(StartMyBootApplication.class, args);
    }
}
```



11) Add the Required Beans Bean Configuration Class.

12) Mark the Documentaion Related Annotations in POJO Classes/Resource Classes.

@ApiModel
@ApiModelProperty

13) Mark the Documentaion Related Annotations in Controllers.

@ApiOperation

14) Start the Application

15) pen the following URL

<http://localhost:12345/swagger-ui.html>

demo7-rest-swagger – Swagger Example

demo7-rest-swagger: Files required

1. MyBootApplication.java	2. Account.java
3. AccountController.java	4. Customer.java
5. CustomerController.java	6. JLCApplicationConfig.java
7. application.properties	8. pom.xml

1. MyBootApplication.java

```
package com.jlccindia.springboot;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

import springfox.documentation.swagger2.annotations.EnableSwagger2;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */

@SpringBootApplication
@EnableSwagger2
public class MyBootApplication {
    public static void main(String as[]) {
        SpringApplication.run(MyBootApplication.class, as);
    }
}
```



2. Account.java

```
package com.jlcindia.springboot;

import io.swagger.annotations.ApiModel;
import io.swagger.annotations.ApiModelProperty;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */

@ApiModel(value="Account contains Accno,AccType,BranchCode,balance ")
public class Account {

    @ApiModelProperty(value="Contains Acc Number")
    private int accno;

    @ApiModelProperty(value="Contains Acc Type")
    private String atype;

    @ApiModelProperty(value="Contains Branch")
    private String bcode;

    @ApiModelProperty(value="Contains Acc Balance")
    private double balance;

    //Constrcutors
    //Setters and Getters
    //toString()
}
```

3. AccountController.java

```
package com.jlcindia.springboot;

import java.util.*;
import org.springframework.web.bind.annotation.*;
import io.swagger.annotations.ApiOperation;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@RestController
public class AccountController {

    @ApiOperation(value = " get the Balance", response = double.class, notes = "Gives Balance of Customer with Accno")
    @GetMapping("/mybal/{accno}")
    public double getBalance(@PathVariable("accno") String accno) {
```



```
System.out.println("AC-getBalance()" + accno);  
return 25000;
```

```
}
```

```
@ApiOperation(value = " get the Account Information", response = Account.class, notes =  
"Gives Complete Account Information with Accno")
```

```
@GetMapping("/myaccount/{accno}")
```

```
public Account getAccountByAccno(@PathVariable("accno") int accno) {  
System.out.println("AC-getAccountByAccno()" + accno);
```

```
Account acc = new Account(accno, "SA", "B99", 29000.0);  
return acc;
```

```
}
```

```
@ApiOperation(value = " get All Accounts Information", response = List.class, notes = "Gives  
All the Accounts Information")
```

```
@GetMapping("/myaccounts")
```

```
public List<Account> getAllCounts() {  
System.out.println("AC-getAllCounts()");
```

```
List<Account> accList = new ArrayList<>();  
accList.add(new Account(101, "SA", "B99", 19000.0));  
accList.add(new Account(102, "SA", "B99", 29000.0));  
accList.add(new Account(103, "SA", "B99", 29000.0));  
return accList;  
}
```

```
@ApiOperation(value = " Adds Account Info", response = Account.class, notes = "Adds  
Accounts Information")
```

```
@PostMapping(value = "/addAccount")
```

```
public Account addAccount(@RequestBody Account acc) {  
System.out.println("AC-addAccount()");
```

```
acc.setAccno(123);  
return acc;  
}
```

```
@ApiOperation(value = " Updates Account Info", response = String.class, notes = "Updates  
Accounts Information")
```

```
@PutMapping(value = "/updateAccount")
```

```
public String updateAccount(@RequestBody Account acc) {  
System.out.println("AC-updateAccount()");
```

```
String msg = "Account Updated Successfully";  
return msg;  
}
```



```
@ApiOperation(value = " Delete Account Info", response = String.class, notes = "Delete Accounts Information")
```

```
@DeleteMapping(value = "/deleteAccount/{myaccno}")
```

```
public String deleteAccount(@PathVariable("myaccno") String accno) {  
    System.out.println("AC-deleteAccount()");
```

```
    String msg = "Account Deleted Successfully";  
    return msg;
```

```
}  
}
```

4. Customer.java

```
package com.jlcindia.springboot;
```

```
import javax.xml.bind.annotation.XmlRootElement;
```

```
import io.swagger.annotations.ApiModel;
```

```
import io.swagger.annotations.ApiModelProperty;
```

```
/*
```

```
 * @Author : Srinivas Dande
```

```
 * @company : Java Learning Center
```

```
 */
```

```
@ApiModel(value = "Customer contains cid,Cname,email,phone,city ")
```

```
@XmlRootElement
```

```
public class Customer {
```

```
    @ApiModelProperty(value = "Contains Cust Id")
```

```
    private int cid;
```

```
    @ApiModelProperty(value = "Contains Cust Name")
```

```
    private String cname;
```

```
    @ApiModelProperty(value = "Contains Email")
```

```
    private String email;
```

```
    @ApiModelProperty(value = "Contains Phone")
```

```
    private long phone;
```

```
    @ApiModelProperty(value = "Contains City")
```

```
    private String city;
```

```
    //Constrcutors
```

```
    //Setters and Getters
```

```
    //toString()
```

```
}
```



5. CustomerController.java

```
package com.jlcindia.springboot;

import java.util.*;
import org.springframework.web.bind.annotation.*;
import io.swagger.annotations.ApiOperation;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@RestController
public class CustomerController {

    @ApiOperation(value = " get the Customer By Cid", response = double.class, notes = "Gives Customer Full Information By CID")
    @GetMapping(value = "/mycustomer/custId/{custId}", produces = "application/xml")
    public Customer getCustomerByCid(@PathVariable("custId") int custId) {
        System.out.println("CC-getCustomerByCid() - " + custId);

        Customer cust = new Customer(custId, "Srinivas", "sri@jlc.com", 12345, "Blore");
        return cust;
    }

    @ApiOperation(value = " get the Customer By Phone", response = Customer.class, notes = "Gives Customer Full Information By Phone")
    @GetMapping(value = "/mycustomer/phone/{phone}", produces = "application/xml")
    public Customer getCustomerByPhone(@PathVariable("phone") long phone) {
        System.out.println("CC-getCustomerByPhone() - " + phone);

        Customer cust = new Customer(109, "Dande", "dande@jlc.com", phone, "Blore");
        return cust;
    }

    @ApiOperation(value = " get the Customer By City", response = List.class, notes = "Gives Customers Full Information By City")
    @GetMapping(value = "/mycustomer/city/{city}", produces = "application/json")
    public List<Customer> getCustomersByCity(@PathVariable("city") String city) {
        System.out.println("CC-getCustomersByCity() - " + city);

        List<Customer> custList = new ArrayList<>();
        custList.add(new Customer(101, "Sri", "sri@jlc.com", 111, city));
        custList.add(new Customer(102, "Vas", "vas@jlc.com", 222, city));
        custList.add(new Customer(103, "SD", "sd@jlc.com", 333, city));
        custList.add(new Customer(104, "DS", "ds@jlc.com", 444, city));
        return custList;
    }
}
```



6. JLCApplicationConfig.java

```
package com.jlcindia.springboot;

import org.springframework.boot.autoconfigure.SpringBootApplication;

import org.springframework.context.annotation.Bean;
import org.springframework.web.servlet.config.annotation.*;
import com.google.common.base.Predicates;
import springfox.documentation.builders.PathSelectors;
import springfox.documentation.builders.RequestHandlerSelectors;
import springfox.documentation.service.ApiInfo;
import springfox.documentation.service.Contact;
import springfox.documentation.spi.DocumentationType;
import springfox.documentation.spring.web.plugins.Docket;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@SpringBootApplication
public class JLCApplicationConfig implements WebMvcConfigurer {

    private ApiInfo getApiDetails() {
        return new ApiInfo("JLC Bank API",
            "Customer and Account API - part of JLCBank", "1.1",
            "Free to use ",
            new Contact("Srinivas Dande", "https://www.jlcindia.com", "sri@jlcindia.com"),
            "API Under Free Licence",
            "https://www.jlcindia.com");
    }

    @Bean
    public Docket api() {
        return new Docket(DocumentationType.SWAGGER_2).select().paths(PathSelectors.any())
            .apis(Predicates.not(RequestHandlerSelectors.basePackage("org.springframework.boot")))
            .build()
            .apiInfo(getApiDetails());
    }

    @Override
    public void addResourceHandlers(ResourceHandlerRegistry registry) {
        //enabling swagger-ui
        registry.addResourceHandler("swagger-ui.html").addResourceLocations("classpath:/META-INF/resources/");
        registry.addResourceHandler("/webjars/**").addResourceLocations("classpath:/META-INF/resources/webjars/");
    }
}
```



8. application.properties

```
spring.application.name=my-jlc-application  
server.port=12345
```

9. pom.xml

```
<project ...>  
  
<modelVersion>4.0.0</modelVersion>  
<groupId>com.jlcindia.springboot</groupId>  
<artifactId>demo7-rest-swagger</artifactId>  
<version>1.0</version>  
<packaging>jar</packaging>  
<name>demo7-rest-swagger</name>  
<description>Spring - Boot - Rest-swagger</description>  
  
<parent>  
<groupId>org.springframework.boot</groupId>  
<artifactId>spring-boot-starter-parent</artifactId>  
<version>2.5.3</version>  
</parent>  
  
<dependencies>  
  
<dependency>  
<groupId>org.springframework.boot</groupId>  
<artifactId>spring-boot-starter-web</artifactId>  
</dependency>  
  
<dependency>  
<groupId>io.springfox</groupId>  
<artifactId>springfox-swagger2</artifactId>  
<version>2.4.0</version>  
</dependency>  
  
<dependency>  
<groupId>io.springfox</groupId>  
<artifactId>springfox-swagger-ui</artifactId>  
<version>2.4.0</version>  
</dependency>  
  
<dependency>  
<groupId>org.springframework.boot</groupId>  
<artifactId>spring-boot-devtools</artifactId>  
</dependency>  
  
</dependencies>
```



```
<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>

</plugins>
</build>
</project>
```

Q1) Who is responsible for Processing REST Request at Server?

Ans:

DispatcherServlet

Q2) Who is responsible for Making REST Client Call?

Ans:

RestTemplate

Q4) Who is responsible for Marshalling/ UnMarshlling Data at Server?

Ans:

DispatcherServlet

Q3) Who is responsible for Marshalling/UnMarshlling Data at Client?

Ans:

RestTemplate

Q5) How RestTemplate will come to know about XML format/JSON Format?

Ans:

Using Response Headers.

Q6) What are the HttpMessageConverters registered with DS and RT?

Ans:

```
ByteArrayHttpMessageConverter
StringHttpMessageConverter
Jaxb2RootElementHttpMessageConverter
MappingJackson2HttpMessageConverter

ResourceHttpMessageConverter
ResourceRegionHttpMessageConverter
SourceHttpMessageConverter
AllEncompassingFormHttpMessageConverter
```



Q7) If we put @DeleteMapping and put the functionality of getting the customer in the implementation, how the annotation @DeleteMapping plays role?

Ans:

You can ...but Incoming Request Http method must be delete.
But Not Recommended to Use like this.

Q8) Can mark getCustomerByCid() method with @DeleteMapping?

Ans:

Same as Q7

Q9) If I don't want to send entire Object ..i.e I want omit some fields in response How can I do this ?

Ans:

make the fields as Transient ...
Transient fields will not be Marshalled or UnMarshalled.

Q10) What is the use of runtime exception and request object as parameter in the following methods?

Ans:

```
ResponseEntity<MyCustomErrorResponse> handleCidNotFound(RuntimeException,  
WebRequest)  
ResponseEntity<MyCustomErrorResponse> handleCidNotFound(RuntimeException)  
ResponseEntity<MyCustomErrorResponse> handleCidNotFound( WebRequest)
```

Q11) How Can I Customize the Message with ResponseEntityExceptionHandler?

Ans:

Yes , You can.

Q12) Can I send exception error message in header section ?

Ans :

Yes , Any Data you can set to Response Headers (in Success case or failure Case)

Q13) Global Exception handler Vs Local Exception handler

Ans:

Consider InvalidPhoneNumber exception to be thrown in multiple places like while adding phone number or updating phone number, everywhere you need to write same piece of code to avoid that global exception handler is written, you just need to throw exception from anywhere but will be handled and relevant message will be sent back to user from centralized place.

Q14) Which exception handler is specific to controller class in REST API CASE?

Ans: Local Exception handler



Q15) What @controllerAdvice indicates to DS?

Ans:

Tells about Centralized place where all the Exceptions are configured

Q16) What happens when thrown exception is not configured?

Ans:

DS will send exception stack to client

Q17) Can I write two methods to Handle Same Exception in One Global Exception handler?

Ans:

No

Q18) Can I write Multiple Global Exception handlers?

Ans:

Yes

Q19) Can I write two methods to handle Same Exception in Two different Global Exception handlers?

Ans:

Yes

Q20) Can I create basecontroller which will have all exception handling?

Ans:

No

Q21) Can we pass multiple exception classes to @ExceptionHandler?

Ans:

Yes , You can

```
@ExceptionHandler({  
    CustomerIDNotFoundException.class,InvalidPhoneNumberException.class})
```

Q22) Can I write controller by extending base controller?

Ans:

Yes , You centralize the common functionality of All the Controllers in
BaseController

```
@RestController  
class CustomerController extends JLCBaseController{  
  
}
```

Q23) What is the use of Swagger?

Ans:

- 1) Rest API Documentation
- 2) Rest API Testing
- 3) Once the Swagger is Ready, Give Swagger Access to Client to consume Rest API.
- 4) Swagger CodeGen Tool can be used for generating Resources at Client App

Q24) What are the IMP annotations in Swagger?

Ans:

- 1) ApiModel (Resource Class)
- 2) ApiModelProperty (Resource Class Property)
- 3) ApiOperation (Controller Operation)

Q25) Is there any provision that we can use Swagger API document to generate java classes using Jackson libraries at client side?

Ans:

Yes, You do this by using Swagger CodeGen Tool.

Q26) Can I Generate the POJO class based on Schema available in Swagger?

Ans:

Yes, You do this by using Swagger CodeGen Tool.

Q27) How do we share schemas to Client so that Client can generate pojo classes?

Ans:

Schemas are available with Swagger API, You can share them

Q28) If we have multiple Rest API running in different Apps then all these are visible in single Swagger UI?

Ans:

No, we can try *SwaggerHub*

Q29) What is SwaggerHub?

Ans:

- SwaggerHub is a platform that supports the idea of having multiple different APIs managed by a single organization or team, and we can use its back end to load these in as needed to our documentation portal.
- By using a tool like SwaggerHub, we can update and define our 'current' specifications to be shown to API consumers, while also supporting ongoing design and development.